



Міністерство освіти і науки України  
Національний технічний університет України “Київський політехнічний  
інститут імені Ігоря Сікорського”  
Факультет інформатики та обчислювальної техніки  
Кафедра інформаційних систем та технологій

Лабораторна робота №8  
З дисципліни «Технології розроблення програмного забезпечення»  
Тема: **«ШАБЛОНИ «COMPOSITE», «FLYWEIGHT», «INTERPRETER»,  
«VISITOR»»**  
Система ведення нотаток

Виконав:  
Студент групи ІА-24  
Шкарніков А. С.

Перевірив:  
Мягкий М. Ю.

Київ-2024

## Зміст

|                                                                     |   |
|---------------------------------------------------------------------|---|
| Тема:.....                                                          | 3 |
| Мета: .....                                                         | 3 |
| Завдання:.....                                                      | 3 |
| Хід роботи .....                                                    | 3 |
| 1. Реалізувати не менше 3-х класів відповідно до обраної теми ..... | 3 |
| 2. Реалізувати один з розглянутих шаблонів за обраною темою .....   | 5 |
| 3. Перевірка патерну .....                                          | 7 |
| Висновки: .....                                                     | 8 |
| Код:.....                                                           | 8 |

**Тема:**

ШАБЛОНИ «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR»

**Мета:**

Ознайомитися з основними шаблонами проектування, такими як «Composite», «Flyweight», «Interpreter», «Visitor», дослідити їхні принципи роботи та навчитися використовувати їх для створення гнучкого та масштабованого програмного забезпечення.

**Завдання:**

В якості нотаток має бути, як мінімум, текст або зображення, додатково можна додати підтримку файлів розміром до 2Мб, збереження посилань на веб-сторінки, збереження html-тексту з коректним його відображенням. Система повинна дозволяти вести нотатки онлайн, заводити блокноти, прикріпляти до нотаток мітки, давати можливість працювати в системі одночасно багатьом користувачам і одному користувачу працювати з різних комп'ютерів. Користувач повинен мати можливість дати доступ до свого блокноту з замітками іншому користувачу з різними правами (тільки перегляд; перегляд та редагування; перегляд, редагування, додавання та видалення нотаток)

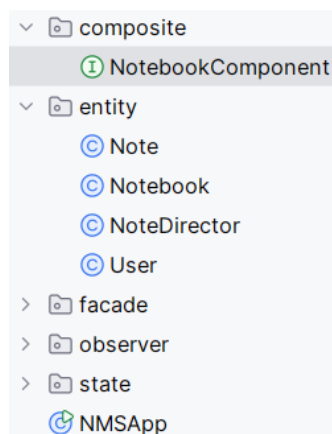
**Хід роботи****1. Реалізувати не менше 3-х класів відповідно до обраної теми**

Рис. 1 — Структура проекту

У ході роботи було створено інтерфейс та оновлені наступні класи:

**1. NotebookComponent**

NotebookComponent є базовим інтерфейсом для всіх елементів системи нотаток у рамках патерну Composite. Він забезпечує спільний контракт для роботи як з простими елементами (листами, наприклад, нотатками), так і зі складними

елементами (композицями, такими як блокноти). Інтерфейс визначає методи для відображення елемента (`display`), управління дочірніми компонентами (`addComponent`, `removeComponent`, `getComponents`) і отримання інформації про елемент (`getName`, `getType`). Це забезпечує уніфіковану взаємодію з різними типами компонентів системи, незалежно від їх складності.

## 2. Note

Note є листовим компонентом у патерні Composite, який представляє окрему нотатку. Він реалізує інтерфейс `NotebookComponent`, зберігаючи інформацію про назву, вміст, тип нотатки (наприклад, текст, зображення чи посилання) і пов'язані з нею теги. Цей клас використовується для створення елементарних компонентів системи, які не можуть містити дочірніх елементів. Основні функції включають метод `display` для відображення вмісту нотатки та роботу з тегами для класифікації й пошуку.

## 3. Notebook

Notebook є композитним компонентом у патерні Composite, який представляє блокнот, що може містити інші блокноти або нотатки. Реалізуючи інтерфейс `NotebookComponent`, цей клас забезпечує можливість додавання, видалення та управління дочірніми елементами, створюючи ієрархічну структуру. Основні функції включають метод `display` для відображення всього вмісту блокноту, метод `findByTag` для пошуку нотаток за тегами та механізм встановлення прав доступу для різних користувачів. Це дозволяє організувати та управляти складними наборами нотаток.

## 2. Реалізувати один з розглянутих шаблонів за обраною темою

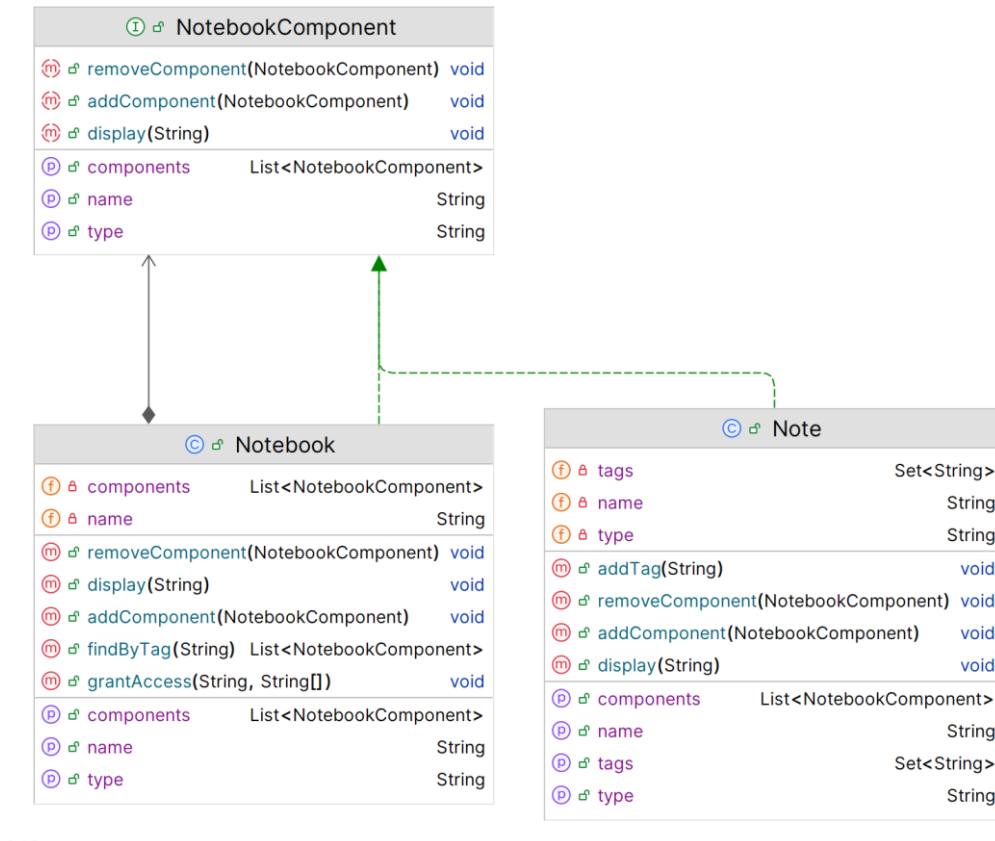


Рис. 2 — Діаграма класів

У даній системі ведення нотаток патерн **Composite** реалізовано для побудови ієрархічної структури, яка дозволяє організовувати нотатки та блокноти. В основі структури лежить інтерфейс `NotebookComponent`, який є спільним контрактом для всіх компонентів системи — як простих (листіків, таких як `Note`), так і складних (комполітів, таких як `Notebook`). Кожен компонент, незалежно від його природи, має уніфікований набір методів для управління та відображення.

Проблеми, які вирішує патерн `Composite`:

1. **Єдиний інтерфейс для різних типів компонентів:** Завдяки `NotebookComponent` система працює однаково як з окремими нотатками, так і з блокнотами, незалежно від їх складності.
2. **Гнучке управління ієрархією:** `Composite` дозволяє легко створювати складну структуру з вкладених блокнотів і нотаток, організовуючи дані в логічну ієрархію.
3. **Масштабованість:** Додавання нових типів компонентів (наприклад, нових видів нотаток або категорій блокнотів) можливе без змін у клієнтському коді.

4. **Рекурсивна обробка:** Система може обробляти всі елементи (нотатки та блокноти) рекурсивно, наприклад, для відображення, пошуку або експорту даних.

Переваги використання патерну Composite:

1. **Простота розширення:** Додавання нових компонентів у систему не потребує змін у вже існуючому коді. Наприклад, можна легко додати новий тип компонента (такого як закріплені нотатки) шляхом реалізації `NotebookComponent`.
2. **Зручна робота з ієрархіями:** Composite дозволяє маніпулювати цілими групами елементів так само, як і окремими компонентами, що знижує складність роботи з багаторівневими структурами.
3. **Кодова уніфікація:** Усі елементи системи мають однаковий інтерфейс, що спрощує реалізацію загальних функцій, таких як відображення, пошук чи управління.
4. **Зниження складності клієнтського коду:** Завдяки рекурсивним методам, клієнту не потрібно самостійно обходити ієрархію компонентів або дбати про їхні типи — всі виклики автоматично застосовуються до всіх вкладених елементів.

### 3. Перевірка патерну

```
public class NMSApp {
    public static void main(String[] args) {
        // Створюємо кореневий блокнот
        Notebook rootNotebook = new Notebook( name: "Root Notebook");

        // Створюємо підблокнот для роботи
        Notebook workNotebook = new Notebook( name: "Work Notes");

        // Створюємо нотатки
        Note textNote = new Note( name: "Meeting Minutes", | content: "Discuss project deadlines", type: "text");
        textNote.addTag("work");
        textNote.addTag("important");

        Note linkNote = new Note( name: "Resources", content: "https://example.com/docs", type: "link");
        linkNote.addTag("reference");

        // Додаємо нотатки до робочого блокноту
        workNotebook.addComponent(textNote);
        workNotebook.addComponent(linkNote);

        // Створюємо особистий блокнот
        Notebook personalNotebook = new Notebook( name: "Personal Notes");
        Note todoNote = new Note( name: "TODO", content: "1. Buy groceries\n2. Call mom", type: "text");
        todoNote.addTag("personal");
        personalNotebook.addComponent(todoNote);

        // Додаємо всі блокноти до кореневого
        rootNotebook.addComponent(workNotebook);
        rootNotebook.addComponent(personalNotebook);

        // Надаємо доступ до робочого блокноту колезі
        workNotebook.grantAccess( userId: "colleague1", ...permissions: "view", "edit");

        // Відображаємо всю структуру
        System.out.println("Full Notebook Structure:");
        rootNotebook.display( indent: "");

        // Шукаємо всі важливі нотатки
        System.out.println("\nImportant Notes:");
        List<NotebookComponent> importantNotes = rootNotebook.findByTag("important");
        for (NotebookComponent note : importantNotes) {
            note.display( indent: "");
        }
    }
}
```

Рис. 3 — Перевірка роботи

У методі main демонструється створення ієрархічної структури системи ведення нотаток з використанням патерну Composite. Спочатку створюється кореневий блокнот Root Notebook, який слугує основним контейнером для всіх інших

блокнотів і нотаток. До нього додається підблокнот Work Notes, в який включаються дві нотатки: текстова нотатка із записами про зустрічі та посилання на ресурси. Кожна нотатка отримує відповідні теги для організації та полегшення пошуку. Після цього створюється ще один блокнот, Personal Notes, в який додається особиста текстова нотатка зі списком справ. Усі ці блокноти додаються до кореневого блокноту, створюючи багаторівневу структуру.

```
Full Notebook Structure:
+ Notebook: Root Notebook
  + Notebook: Work Notes
    - Note: Meeting Minutes (Type: text)
      Content: Discuss project deadlines
      Tags: important, work
    - Note: Resources (Type: link)
      Content: https://example.com/docs
      Tags: reference
  + Notebook: Personal Notes
    - Note: TODO (Type: text)
      Content: 1. Buy groceries
2. Call mom
   Tags: personal

Important Notes:
- Note: Meeting Minutes (Type: text)
  Content: Discuss project deadlines
  Tags: important, work
```

Рис. 4 — Результат роботи

Далі демонструються основні можливості системи. Для робочого блокноту Work Notes надається доступ користувачу colleague1 із правами перегляду та редагування. Після цього відображається вся структура блокнотів та нотаток, використовуючи метод display, який рекурсивно проходить ієрархію. Нарешті, виконується пошук нотаток за тегом "important" у всій структурі за допомогою методу findByTag. Знайдені нотатки відображаються в консолі, демонструючи, як система дозволяє організовувати, відображати та шукати інформацію.

## Висновки:

У цій лабораторній роботі ми реалізували патерн Composite, щоб ефективно працювати з системи ведення нотаток. Це дозволяє спростити дизайн системи, зробити її більш зрозумілою та зручною для подальшого розширення.

## Код:

[https://github.com/Atl4sDev/TRPZ\\_labs](https://github.com/Atl4sDev/TRPZ_labs)